Machine Learning Block 2 Lab 1

Nazli Bilgic (nazbi056), Anastasia Piadi (anapi501), Max Björklund (maxbj107)

2024-01-15

Statement of Contribution

Every group member provided solutions for both assignments.

1. ENSEMBLE METHODS

Your task is to learn some random forests using the function randomForest from the R package randomForest. The training data is produced by running the following R code:

The task is therefore classifying Y from X1 and X2, where Y is binary and X1 and X2 continuous. You should learn a random forest with 1, 10 and 100 trees, which you can do by setting the argument ntree to the appropriate value. Use nodesize = 25 and keep.forest = TRUE. The latter saves the random forest learned. You need it because you should also compute the misclassification error in the following test dataset (use the function predict for this purpose):

- *Repeat the procedure above for 1000 training datasets of size 100 and report the mean and variance of the misclassification errors. In other words, create 1000 training datasets of size 100, learn a random forest from each dataset, and compute the mis-classification error in the same test dataset of size 1000. Report results for when the random forest has 1, 10 and 100 trees.*
- *Repeat the exercise above but this time use the condition (x1<0.5) instead of (x1<x2) when producing the training and test datasets.*
- *Repeat the exercise above but this time use the condition ((x1<0.5 & x2<0.5) | (x1>0.5 & x2>0.5)) instead of (x1<x2) when producing the training and test datasets. Unlike above, use nodesize = 12 for this exercise.*

Code is in the appendix.

```
## randomForest 4.7-1.1
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
## [1] "Mean misclassification error for 1 tree(x1<x2): 0.198699 Variance: 0.00265955395295295"
```

```
## [1] "Mean misclassification error for 10 trees(x1<x2): 0.134026 Variance: 0.000801775099099099"
```

```
## [1] "Mean misclassification error for 100 trees(x1<x2): 0.111507 Variance: 0.00072671866966967"
```

```
## [1] "Mean error-1-condition(x1<0.5): 0.092951 Variance-1-condition(x1<0.5): 0.0172945151141141"
```

```
## [1] "Mean error-10-condition(x1<0.5): 0.013715 Variance-10-condition(x1<0.5): 0.000463973748748749"

## [1] "Mean error-100-condition(x1<0.5): 0.006406 Variance-100-condition(x1<0.5): 6.65737377377377e-05

## [1] "Mean error-1-(node 12): 0.251183 Variance-1-(node 12): 0.0132559394504505"

## [1] "Mean error-10-(node 12): 0.122962 Variance-10-(node 12): 0.00324340796396396"

## [1] "Mean error-100-(node 12): 0.076919 Variance-100-(node 12): 0.0014101565955956"
```

Answer the following questions:

What happens with the mean error rate when the number of trees in the random forest grows? Why?

When the number of trees in random forest grows, mean error decrease. As more trees are added, the model becomes better at capturing different aspects of the data, leading to improved accuracy. Predictions converge more likely to the true pattern.

Every tree in a random forest is constructed using a unique subset of the data and selects a random subset of features for splitting. This variation ensures that each tree may understand different parts of the data. The collective integration of these varied insights offers a wide understanding of the data, which can result to a reduction in the overall error rates.

The third dataset represents a slightly more complicated classification problem than the first one. Still, you should get better performance for it when using sufficient trees in the random forest. Explain why you get better performance.

First one represents a simple linear boundary. A single tree or just a few trees can effectively and simply represent this line. In this case the trees just have to find the linear relationship between x1 and x2.

The third dataset represents a more complex classification problem. Labels are divided according to x1 and x2 being below 0.5 or above 0.5. This creates a division with four quadrants which is a non-linear classification problem. Although the boundary is not a straight line, it is a combination of multiple linear lines which align with our features axis.

The combination of multiple trees allows the random forest to capture more complex patterns in the third dataset. Each tree in a random forest makes decisions according to feature values, creating decision boundaries. In a quadrant space, trees can create splits that adapt to this complexity. Taking the nature of random forest into account, such a problem of combining simple relationships fits the true classification of the data, as its basic splitting mechanism aligns with the structure of the decision boundary in the data.

Random forests collect the result predictions of many trees. This means that even one tree can partially capture a quadrant structure, combination of all the trees in the forest will result in a more accurate overall boundary. Also, in the third dataset node size is 12, smaller node size allows more complex models.

2. Mixture Models

Your task is to implement the EM algorithm for Bernoulli mixture model. Please use the R template below to solve the assignment. Then, use your implementation to show what happens when your mixture model has too few and too many clusters, i.e. set M = 2, 3, 4 and compare results. Please provide a short explanation as well. A Bernoulli mixture model is

$$p(x) = \sum_{m=1}^{M} \pi_m \mathrm{Bern}(x|\mu_m)$$

where x=$x_1, x_2, \ldots, x_D$ *is a D dimensional binary random vector* $\pi_m = p(y = m)$ *and*

$$\mathrm{Bern}(x|\mu_m) = \prod_{d=1}^{D} \mu_{m,d}^{x_d}(1 - \mu_{m,d})^{(1-x_d)}$$

where $\mu_m = (\mu_{m,1}, \ldots, \mu_{m,D})$ *is a D-dimensional vector of probabilities. As usual, the log likelihood of the dataset* $\{x_i\}_{i=1}^n$ *is*

$$\sum_{i=1}^{n} \log p(x_i)$$

Finally, in the EM algorithm, the parameter updates for the Bernoulli mixture model are the same as for the Gaussian mixture model

$$\hat{\pi}_m = \frac{1}{n} \sum_{i=1}^{n} w_i(m),$$

$$\hat{\mu}_m = \frac{1}{\sum_{i=1}^{n} w_i(m)} \sum_{i=1}^{n} w_i(m) x_i,$$

where $w_i(m) = p(y_i = m|x_i, \hat{\theta})$.

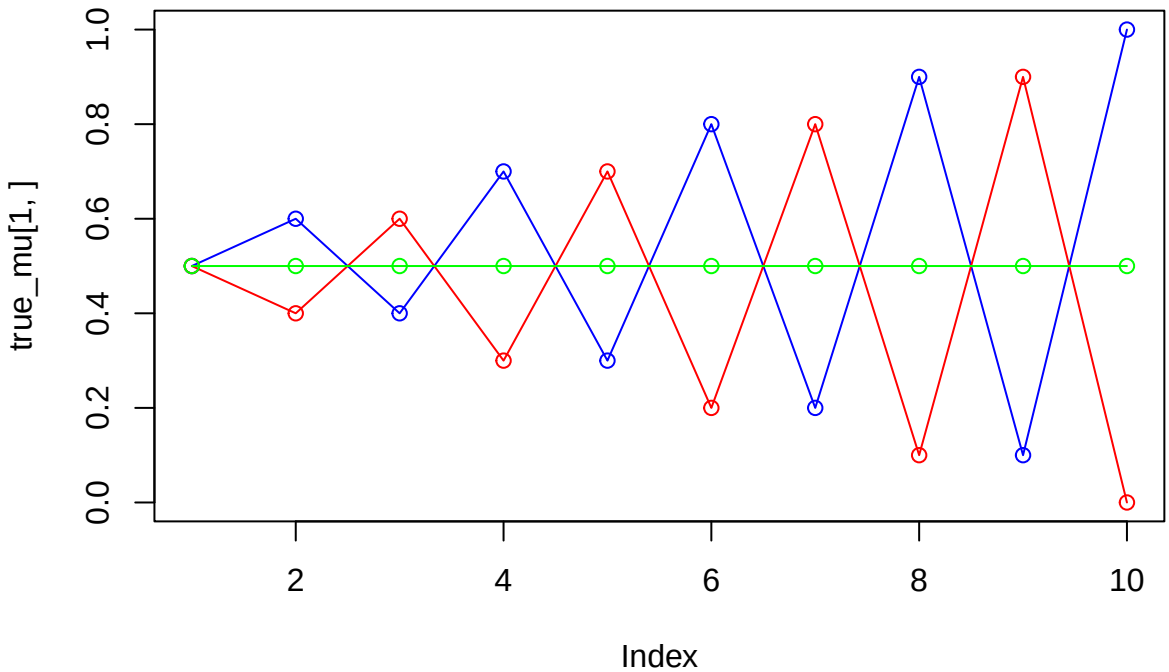
Implementation The EM algorithm consists of two main steps applied:

E-step (Expectation step): Calculate the probabilities (weights w) that each data point belongs to each cluster based on the current estimates of the parameters (mixing coefficients pi and conditional distributions mu).

M-step (Maximization step): Update the parameters to maximize the likelihood of the data given these newly estimated weights.

The likelihood computation after the E-step is also needed to monitor the convergence of the algorithm.

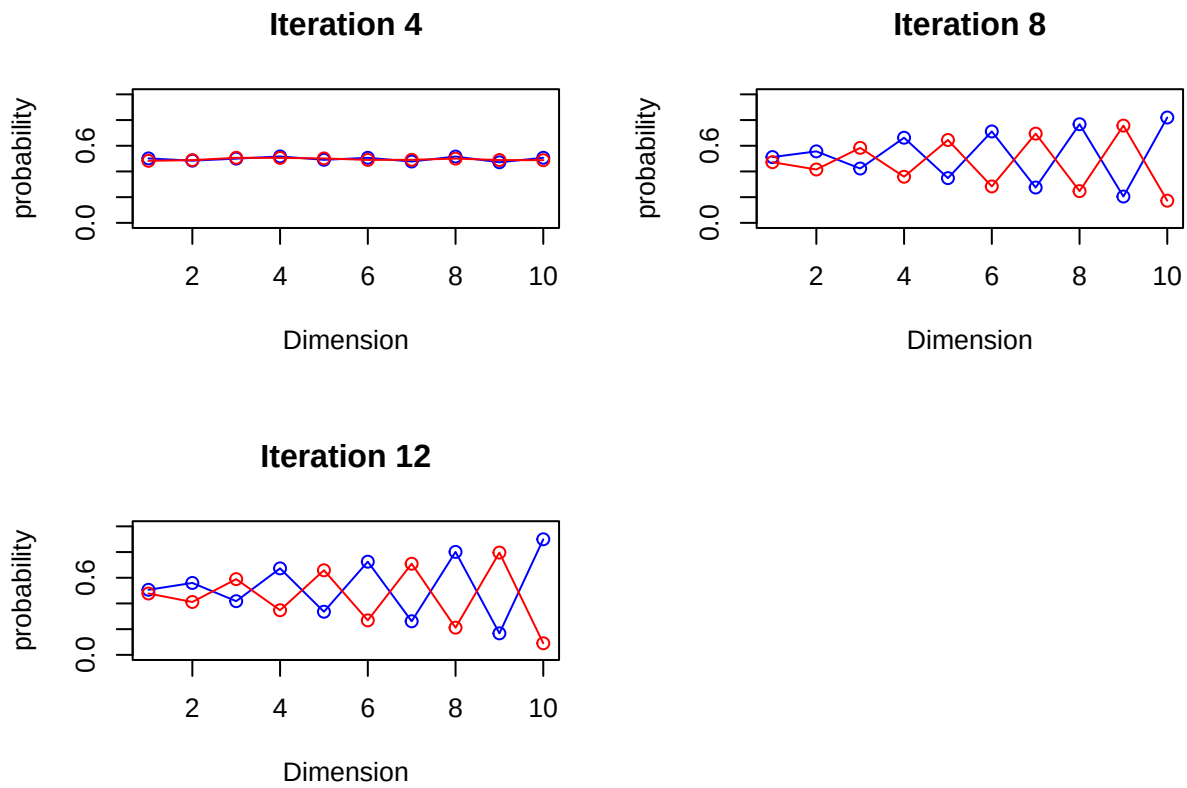
Training Data



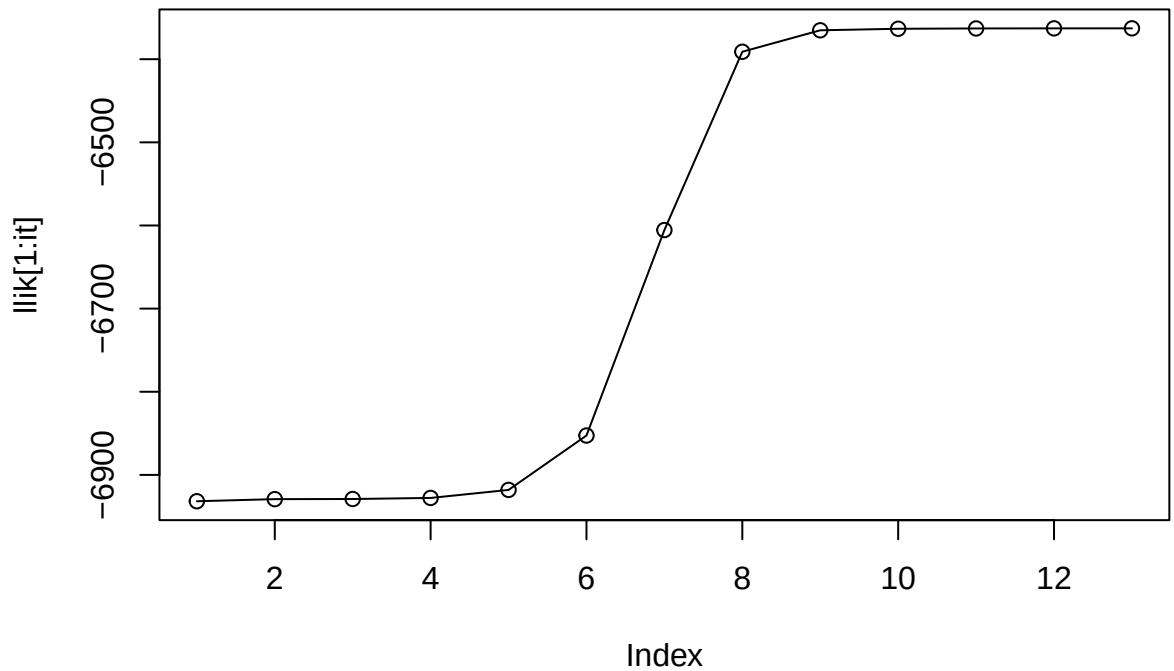
M = 2

```
## Initial values of pi:
##  0.4983671 0.5016329

## Initial values of mu:

##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## [1,] 0.5083050 0.4905939 0.5053502 0.4996011 0.4917676 0.5072589 0.4999321
## [2,] 0.4907538 0.5003319 0.5063692 0.4999025 0.4935788 0.5075828 0.4987468
##           [,8]      [,9]     [,10]
## [1,] 0.5070701 0.5025178 0.4948291
## [2,] 0.5045254 0.5072954 0.4959821

## Log likelihood at iteration 1 : -6931.689
## Log likelihood at iteration 2 : -6929.172
## Log likelihood at iteration 3 : -6929.005

## Log likelihood at iteration 4 : -6927.736
## Log likelihood at iteration 5 : -6918.072
## Log likelihood at iteration 6 : -6852.641
## Log likelihood at iteration 7 : -6605.494
```

```
## Log likelihood at iteration 8 : -6391.035
## Log likelihood at iteration 9 : -6365.216
## Log likelihood at iteration 10 : -6363.38
## Log likelihood at iteration 11 : -6363.019

## Log likelihood at iteration 12 : -6362.918
## Log likelihood at iteration 13 : -6362.887
```



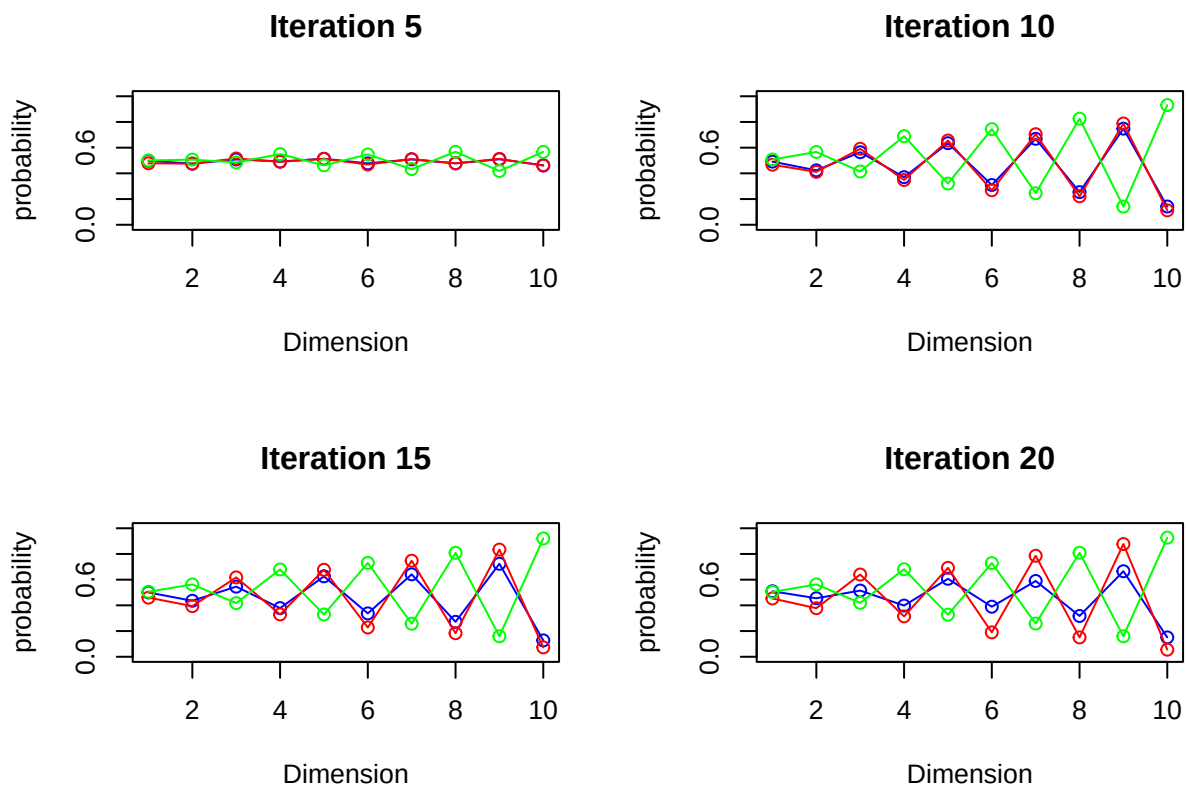

```
## Final estimated pi:

## [1] 0.5024681 0.4975319

## Final estimated mu:

##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## [1,] 0.5062684 0.5598666 0.4177494 0.6730026 0.3353346 0.7247480 0.2616073
## [2,] 0.4775901 0.4114006 0.5890964 0.3473901 0.6582594 0.2690024 0.7085991
##           [,8]      [,9]      [,10]
## [1,] 0.8007841 0.1678141 0.90116765
## [2,] 0.2123112 0.7952831 0.08882252
```



M = 3

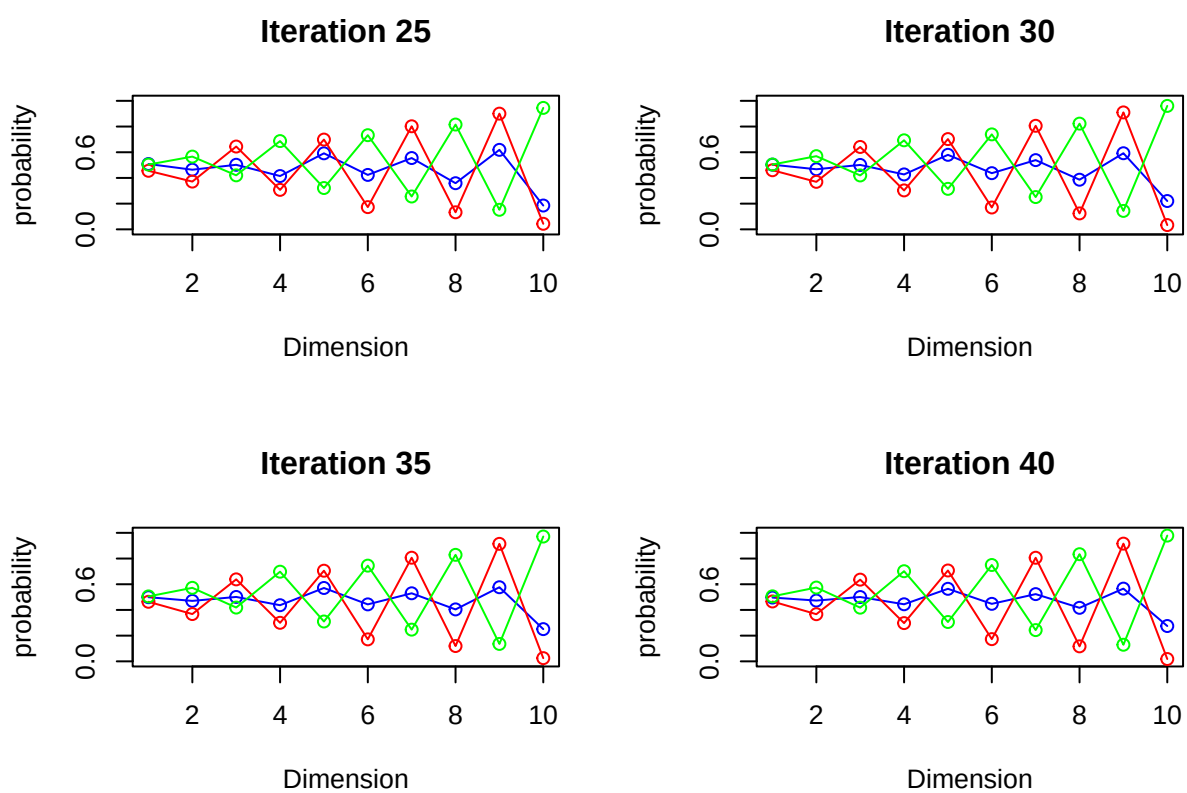
```
## Initial values of pi:
##  0.3324767 0.3345183 0.333005

## Initial values of mu:

##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## [1,] 0.5077225 0.4991296 0.4933274 0.4965019 0.5001845 0.5045541 0.5097947
## [2,] 0.4900227 0.4978241 0.4992499 0.4977629 0.4980497 0.4935793 0.5090332
## [3,] 0.5041496 0.5028909 0.4977966 0.5039709 0.5008812 0.4945293 0.4996912
##           [,8]      [,9]     [,10]
## [1,] 0.4906907 0.4930475 0.5047137
## [2,] 0.4990746 0.4965350 0.5093083
## [3,] 0.5058601 0.4901198 0.4937542

## Log likelihood at iteration 1 : -6931.779
## Log likelihood at iteration 2 : -6929.141
## Log likelihood at iteration 3 : -6928.567
## Log likelihood at iteration 4 : -6924.291

## Log likelihood at iteration 5 : -6893.244
## Log likelihood at iteration 6 : -6731.712
## Log likelihood at iteration 7 : -6456.9
```

```
## Log likelihood at iteration 8 : -6378.899
## Log likelihood at iteration 9 : -6368.98

## Log likelihood at iteration 10 : -6365.654
## Log likelihood at iteration 11 : -6363.883
## Log likelihood at iteration 12 : -6362.655
## Log likelihood at iteration 13 : -6361.604
## Log likelihood at iteration 14 : -6360.566

## Log likelihood at iteration 15 : -6359.465
## Log likelihood at iteration 16 : -6358.281
## Log likelihood at iteration 17 : -6357.037
## Log likelihood at iteration 18 : -6355.785
## Log likelihood at iteration 19 : -6354.586
```



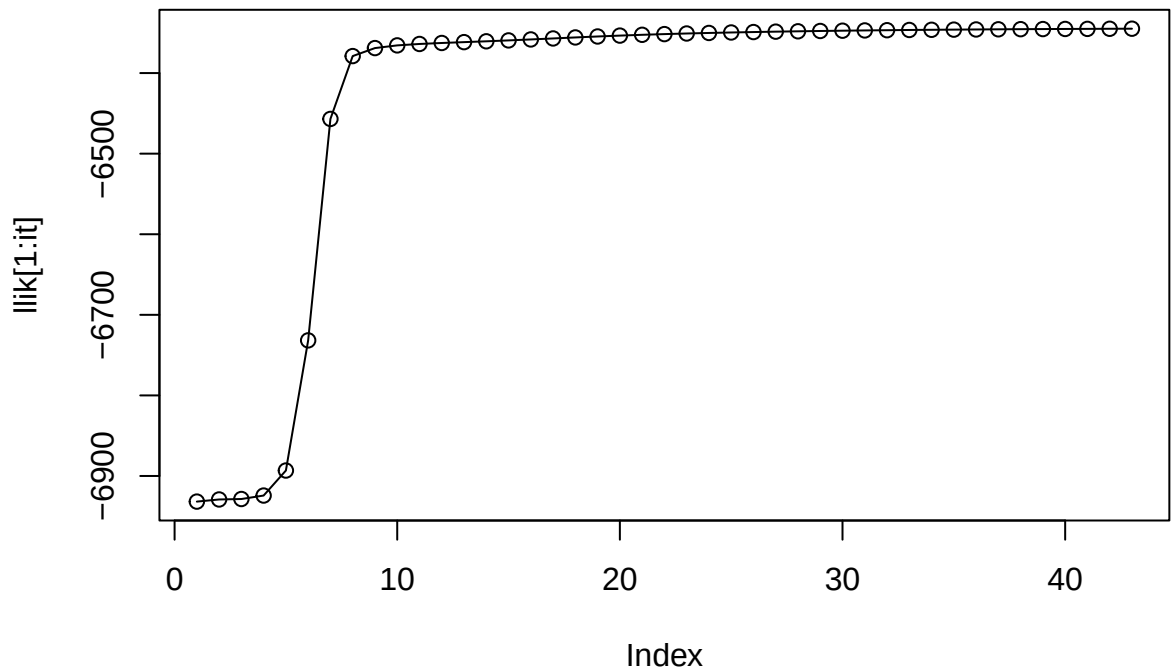

```
## Log likelihood at iteration 20 : -6353.485
## Log likelihood at iteration 21 : -6352.504
## Log likelihood at iteration 22 : -6351.641
## Log likelihood at iteration 23 : -6350.883
## Log likelihood at iteration 24 : -6350.214

## Log likelihood at iteration 25 : -6349.619
## Log likelihood at iteration 26 : -6349.085
## Log likelihood at iteration 27 : -6348.604
## Log likelihood at iteration 28 : -6348.169
## Log likelihood at iteration 29 : -6347.775
```

```
## Log likelihood at iteration 30 : -6347.418
## Log likelihood at iteration 31 : -6347.094
## Log likelihood at iteration 32 : -6346.8
## Log likelihood at iteration 33 : -6346.534
## Log likelihood at iteration 34 : -6346.294

## Log likelihood at iteration 35 : -6346.076
## Log likelihood at iteration 36 : -6345.879
## Log likelihood at iteration 37 : -6345.701
## Log likelihood at iteration 38 : -6345.54
## Log likelihood at iteration 39 : -6345.395
```




```
## Log likelihood at iteration 40 : -6345.264
## Log likelihood at iteration 41 : -6345.145
## Log likelihood at iteration 42 : -6345.038
## Log likelihood at iteration 43 : -6344.941

## Final estimated pi:

## [1] 0.3168444 0.2736185 0.4095371

## Final estimated mu:

##            [,1]       [,2]       [,3]       [,4]       [,5]       [,6]       [,7]
```

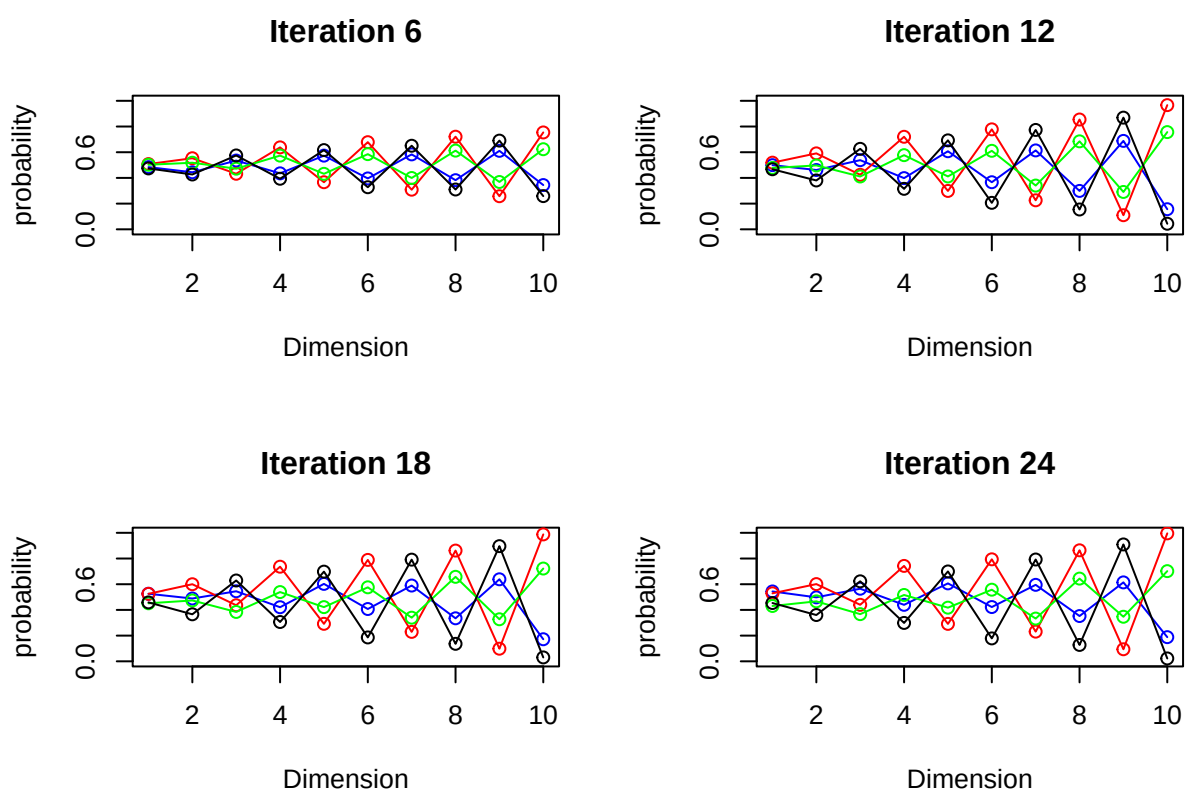
```
## [1,] 0.4951530 0.4738535 0.4998068 0.4476599 0.5611934 0.4494763 0.5199745
## [2,] 0.4670637 0.3666418 0.6347022 0.2966181 0.7076563 0.1737562 0.8056901
## [3,] 0.5062210 0.5751425 0.4174780 0.7032361 0.3041511 0.7521737 0.2412414
##           [,8]      [,9]      [,10]
## [1,] 0.4221528 0.5609184 0.28667769
## [2,] 0.1159810 0.9163045 0.01487476
## [3,] 0.8363313 0.1258941 0.98183476
```



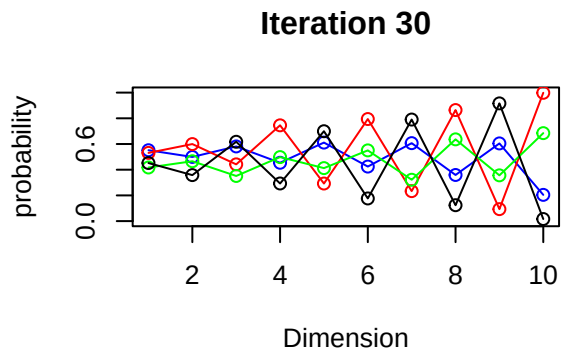
$\mathbf{M = 4}$

```
## Initial values of pi:
##  0.2491106 0.2506403 0.2495065 0.2507425

## Initial values of mu:

##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## [1,] 0.4991296 0.4933274 0.4965019 0.5001845 0.5045541 0.5097947 0.4906907
## [2,] 0.4978241 0.4992499 0.4977629 0.4980497 0.4935793 0.5090332 0.4990746
## [3,] 0.5028909 0.4977966 0.5039709 0.5008812 0.4945293 0.4996912 0.5058601
## [4,] 0.4974021 0.4972325 0.5073759 0.5080831 0.5023485 0.4926806 0.5056439
##           [,8]      [,9]     [,10]
## [1,] 0.4930475 0.5047137 0.4900227
## [2,] 0.4965350 0.5093083 0.5041496
## [3,] 0.4901198 0.4937542 0.5036367
## [4,] 0.4985840 0.5085455 0.5054649
```

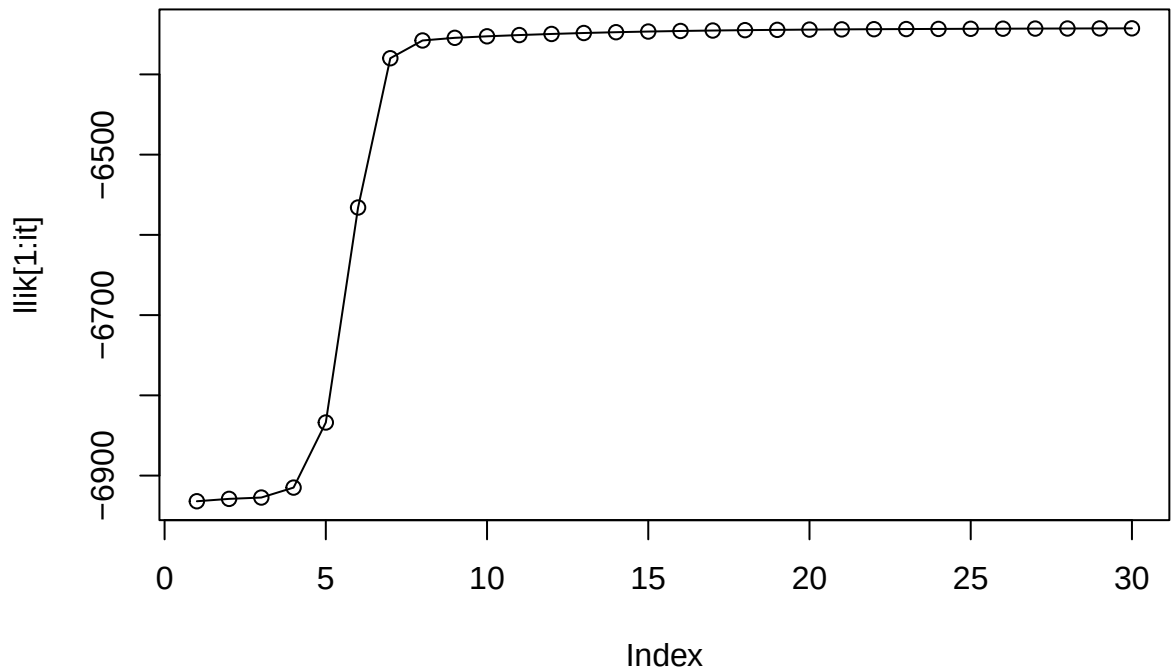
```
## Log likelihood at iteration 1 : -6931.948
## Log likelihood at iteration 2 : -6928.997
## Log likelihood at iteration 3 : -6927.337
## Log likelihood at iteration 4 : -6914.933
## Log likelihood at iteration 5 : -6833.824

## Log likelihood at iteration 6 : -6565.912
## Log likelihood at iteration 7 : -6379.842
## Log likelihood at iteration 8 : -6357.718
## Log likelihood at iteration 9 : -6354.426
## Log likelihood at iteration 10 : -6352.551
## Log likelihood at iteration 11 : -6350.995

## Log likelihood at iteration 12 : -6349.625
## Log likelihood at iteration 13 : -6348.433
## Log likelihood at iteration 14 : -6347.422
## Log likelihood at iteration 15 : -6346.582
## Log likelihood at iteration 16 : -6345.893
## Log likelihood at iteration 17 : -6345.331

## Log likelihood at iteration 18 : -6344.87
## Log likelihood at iteration 19 : -6344.49
## Log likelihood at iteration 20 : -6344.174
## Log likelihood at iteration 21 : -6343.908
## Log likelihood at iteration 22 : -6343.682
## Log likelihood at iteration 23 : -6343.488
```



```
## Log likelihood at iteration 24 : -6343.319
## Log likelihood at iteration 25 : -6343.17
## Log likelihood at iteration 26 : -6343.037
## Log likelihood at iteration 27 : -6342.918
## Log likelihood at iteration 28 : -6342.809
## Log likelihood at iteration 29 : -6342.708

## Log likelihood at iteration 30 : -6342.615
```




```
## Final estimated pi:

## [1] 0.2159257 0.3192132 0.1894910 0.2753701

## Final estimated mu:

##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## [1,] 0.5515145 0.4993172 0.5810061 0.4539429 0.6110702 0.4243963 0.6066287
## [2,] 0.5307404 0.5993952 0.4416433 0.7454390 0.2923581 0.7943436 0.2331247
## [3,] 0.4171721 0.4658535 0.3513075 0.4981286 0.4123317 0.5502536 0.3217493
## [4,] 0.4519160 0.3579717 0.6173432 0.2928322 0.6994097 0.1762315 0.7903116
##           [,8]       [,9]      [,10]
## [1,] 0.3589064 0.60456519 0.20418236
## [2,] 0.8641815 0.09308165 0.99846793
## [3,] 0.6362972 0.35611316 0.68464768
## [4,] 0.1237324 0.91609681 0.01617136
```



Analysis

For M=3, estimated 'mu' values align more closely with the true 'mu' values and capture the true values without overfit or underfitting. This is beacuse, the number of estimated clusters match the true number of the clusters. The log-likelihood plot shows increase and then stability. This shows that the model fits the data well and comes to a point where additional iterations do not improve the fit anymore.

For M=2 the algorithm tries to approximate the data generated by three clusters with only two clusters. This cause underfit(model is too simple to capture the complexity of the data) and all the data can not be captured with its complete properties.

As M increases model complexity also increase. For, M=4 with four clusters, in this case we have more clusters than the true number of clusters. This has a risk of overfitting.

Appendix

Chunk Label: Question 1

```
library(randomForest)

#  training dataset
set.seed(1234)
x1 <- runif(100)
```

```r
x2 <- runif(100)
trdata <- cbind(x1, x2)
y <- as.numeric(x1 < x2)
trlabels <- as.factor(y)

# train the Random Forests
rf_1 <- randomForest(trdata, trlabels, ntree = 1, nodesize = 25, keep.forest = TRUE)
rf_10 <- randomForest(trdata, trlabels, ntree = 10, nodesize = 25, keep.forest = TRUE)
rf_100 <- randomForest(trdata, trlabels, ntree = 100, nodesize = 25, keep.forest = TRUE)

# test dataset
x1 <- runif(1000)
x2 <- runif(1000)
tedata <- cbind(x1, x2)
y <- as.numeric(x1 < x2)
telabels <- as.factor(y)

#plot(x1, x2, col = (y + 1))

# Prediction and misclassification Error on Test Data
pred_test_rf_1 <- predict(rf_1, tedata)
pred_test_rf_10 <- predict(rf_10, tedata)
pred_test_rf_100 <- predict(rf_100, tedata)

misclass_error_test_rf_1 <- mean(pred_test_rf_1 != telabels)
misclass_error_test_rf_10 <- mean(pred_test_rf_10 != telabels)
misclass_error_test_rf_100 <- mean(pred_test_rf_100 != telabels)


# 1000 training datasets
errors_rf_1 <- numeric(1000)
errors_rf_10 <- numeric(1000)
errors_rf_100 <- numeric(1000)

for (i in 1:1000) {
  set.seed(i)  # Optional for reproducibility
  x1 <- runif(100)
  x2 <- runif(100)
  trdata <- cbind(x1, x2)
  y <- as.numeric(x1 < x2)  # or use x1 < 0.5 as per your requirement
  trlabels <- as.factor(y)

  # Train random forests
  rf_1 <- randomForest(trdata, trlabels, ntree = 1, nodesize = 25, keep.forest = TRUE)
  rf_10 <- randomForest(trdata, trlabels, ntree = 10, nodesize = 25, keep.forest = TRUE)
  rf_100 <- randomForest(trdata, trlabels, ntree = 100, nodesize = 25, keep.forest = TRUE)

  # Prediction and errors on test data
  errors_rf_1[i] <- mean(predict(rf_1, tedata) != telabels)
  errors_rf_10[i] <- mean(predict(rf_10, tedata) != telabels)
  errors_rf_100[i] <- mean(predict(rf_100, tedata) != telabels)
}
```

```r
# Calculation of mean and variance of errors
mean_error_rf_1 <- mean(errors_rf_1)
var_error_rf_1 <- var(errors_rf_1)

mean_error_rf_10 <- mean(errors_rf_10)
var_error_rf_10 <- var(errors_rf_10)

mean_error_rf_100 <- mean(errors_rf_100)
var_error_rf_100 <- var(errors_rf_100)

print(paste("Mean misclassification error for 1 tree(x1<x2):",
            mean_error_rf_1, "Variance:", var_error_rf_1))
print(paste("Mean misclassification error for 10 trees(x1<x2):",
            mean_error_rf_10, "Variance:", var_error_rf_10))
print(paste("Mean misclassification error for 100 trees(x1<x2):",
            mean_error_rf_100, "Variance:", var_error_rf_100))




#Repeat the exercise above but this time use the condition (x1<0.5) instead of (x1<x2) when producing t

set.seed(1234)

#  test data
x1 <- runif(1000)
x2 <- runif(1000)
tedata <- cbind(x1, x2)
y<- as.numeric(x1 < 0.5)
telabels <- as.factor(y)

#  vectors to store misclassification errors
errors_rf_1 <- numeric(1000)
errors_rf_10 <- numeric(1000)
errors_rf_100 <- numeric(1000)

for (i in 1:1000) {
  x1 <- runif(100)
  x2 <- runif(100)
  trdata <- cbind(x1, x2)
  y <- as.numeric(x1 < 0.5)
  trlabels <- as.factor(y)

  # Train random forests
  rf_1 <- randomForest(trdata, trlabels, ntree = 1, nodesize = 25, keep.forest = TRUE)
  rf_10 <- randomForest(trdata, trlabels, ntree = 10, nodesize = 25, keep.forest = TRUE)
  rf_100 <- randomForest(trdata, trlabels, ntree = 100, nodesize = 25, keep.forest = TRUE)

  # Predictions and errors on test data
  errors_rf_1[i] <- mean(predict(rf_1, tedata) != telabels)
  errors_rf_10[i] <- mean(predict(rf_10, tedata) != telabels)
  errors_rf_100[i] <- mean(predict(rf_100, tedata) != telabels)
}
```

```r
mean_error_rf_1 <- mean(errors_rf_1)
var_error_rf_1 <- var(errors_rf_1) #variance error
mean_error_rf_10 <- mean(errors_rf_10)
var_error_rf_10 <- var(errors_rf_10)
mean_error_rf_100 <- mean(errors_rf_100)
var_error_rf_100 <- var(errors_rf_100)

print(paste("Mean error-1-condition(x1<0.5):",
            mean_error_rf_1, "Variance-1-condition(x1<0.5):",
            var_error_rf_1))
print(paste("Mean error-10-condition(x1<0.5):",
            mean_error_rf_10, "Variance-10-condition(x1<0.5):",
            var_error_rf_10))
print(paste("Mean error-100-condition(x1<0.5):",
            mean_error_rf_100, "Variance-100-condition(x1<0.5):",
            var_error_rf_100))

mean_errors <- c(mean_error_rf_1, mean_error_rf_10, mean_error_rf_100)
variance_errors <- c(var_error_rf_1, var_error_rf_10, var_error_rf_100)


#Repeat the exercise above but this time use the condition ((x1<0.5 & x2<0.5) | (x1>0.5 & x2>0.5))
#instead of (x1<x2) when producing the training and test datasets. Unlike above, use nodesize = 12 for

set.seed(1234)

x1 <- runif(1000)
x2 <- runif(1000)
tedata <- cbind(x1, x2)
y <- as.numeric((x1 < 0.5 & x2 < 0.5) | (x1 > 0.5 & x2 > 0.5))
telabels <- as.factor(y)

errors_rf_1 <- numeric(1000)
errors_rf_10 <- numeric(1000)
errors_rf_100 <- numeric(1000)

for (i in 1:1000) {
  x1 <- runif(100)
  x2 <- runif(100)
  trdata <- cbind(x1, x2)
  y <- as.numeric((x1 < 0.5 & x2 < 0.5) | (x1 > 0.5 & x2 > 0.5))
  trlabels <- as.factor(y)

  # nodesize = 12
  rf_1_node12 <- randomForest(trdata, trlabels, ntree = 1, nodesize = 12, keep.forest = TRUE)
  rf_10_node12 <- randomForest(trdata, trlabels, ntree = 10, nodesize = 12, keep.forest = TRUE)
  rf_100_node12 <- randomForest(trdata, trlabels, ntree = 100, nodesize = 12, keep.forest = TRUE)

  errors_rf_1[i] <- mean(predict(rf_1_node12, tedata) != telabels)
  errors_rf_10[i] <- mean(predict(rf_10_node12, tedata) != telabels)
  errors_rf_100[i] <- mean(predict(rf_100_node12, tedata) != telabels)
}
```

```r
mean_error_rf_1 <- mean(errors_rf_1)
var_error_rf_1 <- var(errors_rf_1)
mean_error_rf_10 <- mean(errors_rf_10)
var_error_rf_10 <- var(errors_rf_10)
mean_error_rf_100 <- mean(errors_rf_100)
var_error_rf_100 <- var(errors_rf_100)

print(paste("Mean error-1-(node 12):",
            mean_error_rf_1, "Variance-1-(node 12):", var_error_rf_1))
print(paste("Mean error-10-(node 12):",
            mean_error_rf_10, "Variance-10-(node 12):", var_error_rf_10))
print(paste("Mean error-100-(node 12):",
            mean_error_rf_100, "Variance-100-(node 12):", var_error_rf_100))
```

Chunk Label: question_2

```r
set.seed(1234567890)
max_it <- 100 # max number of EM iterations
min_change <- 0.1 # min change in log lik between two consecutive iterations
n=1000 # number of training points
D=10 # number of dimensions
x <- matrix(nrow=n, ncol=D) # training data

true_pi <- vector(length = 3) # true mixing coefficients
true_mu <- matrix(nrow=3, ncol=D) # true conditional distributions
true_pi=c(1/3, 1/3, 1/3)
true_mu[1,]=c(0.5,0.6,0.4,0.7,0.3,0.8,0.2,0.9,0.1,1)
true_mu[2,]=c(0.5,0.4,0.6,0.3,0.7,0.2,0.8,0.1,0.9,0)
true_mu[3,]=c(0.5,0.5,0.5,0.5,0.5,0.5,0.5,0.5,0.5,0.5)
plot(true_mu[1,], type="o", col="blue", ylim=c(0,1))
points(true_mu[2,], type="o", col="red")
points(true_mu[3,], type="o", col="green")
# Producing the training data
for(i in 1:n) {
  m <- sample(1:3,1,prob=true_pi)
  for(d in 1:D) {
    x[i,d] <- rbinom(1,1,true_mu[m,d])
  }
}
```

Chunk Label: question2_M_2

```r
set.seed(1234567)

M=2 # number of clusters
w <- matrix(nrow=n, ncol=M) # weights
pi <- vector(length = M) # mixing coefficients
mu <- matrix(nrow=M, ncol=D) # conditional distributions
llik <- vector(length = max_it) # log likelihood of the EM iterations
```

```r
# Random initialization of the parameters
pi <- runif(M,0.49,0.51)
pi <- pi / sum(pi)
for(m in 1:M) {
  mu[m,] <- runif(D,0.49,0.51)
}
cat("Initial values of pi:\n", pi, "\n")
cat("Initial values of mu:\n")
print(mu)
par(mfrow = c(2, 2))

for(it in 1:max_it) {


  if( (it %% 4) == 0 ){
     # Plot for the number of clusters defined by M
  colors <- c("blue", "red", "green", "black")
    plot(mu[1,], type="o", col=colors[1], ylim=c(0,1), xlab="Dimension", ylab = "probability", main = pa
    if(M >= 2) points(mu[2,], type="o", col=colors[2])
  if(M >= 3) points(mu[3,], type="o", col=colors[3])
  if(M >= 4) points(mu[4,], type="o", col=colors[4])
  Sys.sleep(0.5)
  }

  # E-step: Computation of the weights
  for (i in 1:n) {
    total_likelihood <- numeric(M)

    for (m in 1:M) {
      # bernoulli likelihood for data point i under mixture component m
      # Probability that the current data point was generated by the current cluster
      bern <- prod(mu[m,]^x[i,] * (1 - mu[m,])^(1 - x[i,]))

      # weighted likelihood
      prob_x <- pi[m] * bern
      w[i, m] <- prob_x # Constructing a matrix w where each row corresponds to a data point and each c
      total_likelihood[m] <- prob_x # Vector is used to store the sums which will be used to normalize
    }

#Normalize the weights so that they sum to 1 for each data point
 # Dividing each unnormalized weight by the sum of all unnormalized weights for that data point
    w[i,] <- w[i,] / sum(total_likelihood)
  }


  # Log likelihood computation
  log_likelihood <- 0

  for(i in 1:n) {
    # initialize probability
    p_xi <- 0

    for(m in 1:M) {
```

```r
      # compute the bernoulli likelihood for data point i under mixture component m
      bernoulli_prob <- prod(mu[m,]^x[i,] * (1 - mu[m,])^(1 - x[i,]))

      # adding mixture component's contribution to the total probability of x_i
      p_xi <- p_xi + pi[m] * bernoulli_prob
    }

    log_likelihood <- log_likelihood + log(p_xi)
  }

  # store the computed log likelihood in the vector for each iteration
  llik[it] <- log_likelihood
  cat("Log likelihood at iteration", it, ":", log_likelihood, "\n")

# After the EM steps,check for convergence by looking at the difference in log likelihood
  if(it > 1 && abs(llik[it] - llik[it - 1]) < min_change) {
    break
  }

# M-step: ML (Maximum Likelihood) parameter estimation from the data and weights
  for (m in 1:M) {
    # Update the mixing coefficients pi[m]
    pi[m] <- sum(w[,m]) / n

    # Update the Bernoulli probabilities mu[m,] for each dimension d
    for (d in 1:D) {
      mu[m, d] <- sum(w[,m] * x[,d]) / sum(w[,m])
    }
  }


}
```

Chunk Label: question_2_plot

```r
cat("Final estimated pi:\n")
pi
cat("Final estimated mu:\n")
mu
plot(llik[1:it], type="o")
```

Chunk Label: Question_2_M_3

```r
set.seed(12345)

M=3 # number of clusters
w <- matrix(nrow=n, ncol=M) # weights
pi <- vector(length = M) # mixing coefficients
mu <- matrix(nrow=M, ncol=D) # conditional distributions
```

```r
llik <- vector(length = max_it) # log likelihood of the EM iterations

# Random initialization of the parameters
pi <- runif(M,0.49,0.51)
pi <- pi / sum(pi)
for(m in 1:M) {
  mu[m,] <- runif(D,0.49,0.51)
}

cat("Initial values of pi:\n", pi, "\n")
cat("Initial values of mu:\n")
print(mu)
par(mfrow = c(2, 2))

for(it in 1:max_it) {

  if( (it %% 5) == 0 ){
     # Plot for the number of clusters defined by M
  colors <- c("blue", "red", "green", "black")
    plot(mu[1,], type="o", col=colors[1], ylim=c(0,1), xlab="Dimension", ylab = "probability", main = pa
    if(M >= 2) points(mu[2,], type="o", col=colors[2])
  if(M >= 3) points(mu[3,], type="o", col=colors[3])
  if(M >= 4) points(mu[4,], type="o", col=colors[4])
  Sys.sleep(0.5)
  }


  # E-step: Computation of the weights
  for (i in 1:n) {
    total_likelihood <- numeric(M)

    for (m in 1:M) {
#bernoulli likelihood for data point i under mixture component m
#Probability that the current data point was generated by the current cluster
      bern <- prod(mu[m,]^x[i,] * (1 - mu[m,])^(1 - x[i,]))

      # weighted likelihood
      prob_x <- pi[m] * bern
# Constructing a matrix w where each row corresponds to a
#data point and each column to a cluster
      w[i, m] <- prob_x
# Vector is used to store the sums which will be used to normalize the probabilities
      total_likelihood[m] <- prob_x
    }

# Normalize the weights so that they sum to 1 for each data point
# Dividing each unnormalized weight by the sum of all unnormalized weights for that data point
    w[i,] <- w[i,] / sum(total_likelihood)
  }


  # Log likelihood computation
  # set loglikel ihood to zero at the beginning
```

```r
  log_likelihood <- 0

  for(i in 1:n) {
    # initialize probability
    p_xi <- 0

    for(m in 1:M) {
  # compute the bernoulli likelihood for data point i under mixture component m
      bernoulli_prob <- prod(mu[m,]^x[i,] * (1 - mu[m,])^(1 - x[i,]))

      # adding mixture component's contribution to the total probability of x_i
      p_xi <- p_xi + pi[m] * bernoulli_prob
    }

    log_likelihood <- log_likelihood + log(p_xi)
  }

  # store the computed log likelihood in the vector for each iteration
  llik[it] <- log_likelihood
  cat("Log likelihood at iteration", it, ":", log_likelihood, "\n")

  # After the EM steps, check for convergence
  #by looking at the difference in log likelihood
  if(it > 1 && abs(llik[it] - llik[it - 1]) < min_change) {
    break # Break the loop if changes are smaller than the threshold
  }

# M-step: ML (Maximum Likelihood) parameter estimation from the data and weights
  for (m in 1:M) {
# Update the mixing coefficients pi[m]
    pi[m] <- sum(w[,m]) / n

 # Update the Bernoulli probabilities mu[m,] for each dimension d
    for (d in 1:D) {
      mu[m, d] <- sum(w[,m] * x[,d]) / sum(w[,m])
    }
  }


}
```

Chunk Label: question_2_final_plot

```r
cat("Final estimated pi:\n")
pi
cat("Final estimated mu:\n")
mu
plot(llik[1:it], type="o")
```

Chunk Label: question2_M_4

```
set.seed(12345)

M=4 # number of clusters
w <- matrix(nrow=n, ncol=M) # weights
pi <- vector(length = M) # mixing coefficients
mu <- matrix(nrow=M, ncol=D) # conditional distributions
llik <- vector(length = max_it) # log likelihood of the EM iterations

# Random initialization of the parameters
pi <- runif(M,0.49,0.51)
pi <- pi / sum(pi)
for(m in 1:M) {
  mu[m,] <- runif(D,0.49,0.51)
}

cat("Initial values of pi:\n", pi, "\n")
cat("Initial values of mu:\n")
print(mu)
par(mfrow = c(2, 2))

for(it in 1:max_it) {

if( (it %% 6) == 0 ){
     # Plot for the number of clusters defined by M
  colors <- c("blue", "red", "green", "black")
    plot(mu[1,], type="o", col=colors[1], ylim=c(0,1), xlab="Dimension", ylab = "probability", main = pa
    if(M >= 2) points(mu[2,], type="o", col=colors[2])
  if(M >= 3) points(mu[3,], type="o", col=colors[3])
  if(M >= 4) points(mu[4,], type="o", col=colors[4])
  Sys.sleep(0.5)
  }


# E-step: Computation of the weights
  for (i in 1:n) {
# M is the number of clusters. numeric vector with size M
    total_likelihood <- numeric(M)

    for (m in 1:M) { # Iterate over each cluster
# bernoulli likelihood for data point i under mixture component m
# Probability that the current data point was generated by the current cluster
      bern <- prod(mu[m,]^x[i,] * (1 - mu[m,])^(1 - x[i,]))

      # weighted likelihood
      prob_x <- pi[m] * bern
# Constructing a matrix w where each row corresponds
#to a data point and each column to a cluster
      w[i, m] <- prob_x
# Vector is used to store the sums which will be used to normalize the probabilities
      total_likelihood[m] <- prob_x
    }
```

```r
#Normalize the weights so that they sum to 1 for each data point
#Dividing each unnormalized weight by the sum of
  #all unnormalized weights for that data point
    w[i,] <- w[i,] / sum(total_likelihood)
  }


  # Log likelihood computation
#start of each iteration set loglikelihood to zero at the beginning
  log_likelihood <- 0


  for(i in 1:n) {
    # initialize probability
    p_xi <- 0

    for(m in 1:M) {
      # compute the bernoulli likelihood for data
      #point i under mixture component m
      bernoulli_prob <- prod(mu[m,]^x[i,] * (1 - mu[m,])^(1 - x[i,]))

      # adding mixture component's contribution
      #to the total probability of x_i
      p_xi <- p_xi + pi[m] * bernoulli_prob
    }

    log_likelihood <- log_likelihood + log(p_xi)
  }

  # store the computed log likelihood in the vector for each iteration
  llik[it] <- log_likelihood
  cat("Log likelihood at iteration", it, ":", log_likelihood, "\n")

  # After the EM steps, check for convergence by looking at
  #the difference in log likelihood
  if(it > 1 && abs(llik[it] - llik[it - 1]) < min_change) {
    break # Break the loop if changes are smaller than the threshold
  }

  # M-step: ML (Maximum Likelihood) parameter estimation from the data and weights
  for (m in 1:M) {
    # Update the mixing coefficients pi[m]
    pi[m] <- sum(w[,m]) / n

    # Update the Bernoulli probabilities mu[m,] for each dimension d
    for (d in 1:D) {
      mu[m, d] <- sum(w[,m] * x[,d]) / sum(w[,m])
    }
  }


}
```

Chunk Label: question_2_log_plot

```
cat("Final estimated pi:\n")
pi
cat("Final estimated mu:\n")
mu
plot(llik[1:it], type="o")
```